Proyecto Fin de Ciclo — ASIR

Fox Engine y Arquitectura Mother Base

Un entorno empresarial autoadministrable orquestado por IA

Unidad Fox Hound

Marcos Bolívar Muñoz

Sebastián Carrillo Medina

Francisco Castillo Martín

Javier Pérez Martín

Mario Romera Braojos

Entrega final — Mayo de 2026

Índice

1. Introducción y contexto del proyecto 4
1.1. Idea y motivación 4
1.2. El equipo: Unidad Fox Hound 4
1.3. Finalidad y alcance 5
2. Visión global de la arquitectura 5
2.1. Esquema general de funcionamiento 5
2.2. Inventario de nodos y su papel 7
2.3. Recursos asignados en Proxmox 7
3. Comunicaciones e interdependencias entre sistemas 8
3.1. Flujo 1 — Petición remota: del móvil al servidor ejecutado 8
3.2. Flujo 2 — Identidad centralizada: login con LDAP 9
3.3. Flujo 3 — Monitorización y autocuración 9
3.4. Flujo 4 — Acceso unificado: DNS interno + Nginx 10
3.5. Mapa de dependencias entre servicios 11
4. Implementación por nodo 12
4.1. pfSense — El Guardián del perímetro 12
Configuración de red e interfaces 12
Reglas de firewall y NAT 12
Cómo se integra con el resto 13
4.2. Mother Base — Cerebro de identidades y monitor 13
OpenLDAP y phpLDAPadmin 13
Docker Swarm — Manager Leader 14
DNS interno 15
Grafana — el "Radar" 15
Cómo se integra con el resto 15
4.3. Fox Engine — El cerebro de IA 15
Migración a inferencia local: Ollama 16
Orquestador Python multihilo 16
Tres capas de seguridad 16
Modos de operación y autocuración 17
Cliente nativo (APK) 17
Conectividad: Tailscale (Mesh) 18
Cómo se integra con el resto 19
4.4. Plant — Servicios de soporte 19
Passbolt — Gestor de contraseñas 19
FreeScout — Helpdesk 20
Script de backup 20
Cómo se integra con el resto 21
4.5. Tanker — Frontend y documentación 21
Nginx como proxy inverso 21

MkDocs — gestión del conocimiento 22
Cómo se integra con el resto 22
5. Mejoras de la entrega final 23
5.1. Mejoras en Mother Base 23
5.2. Acceso remoto Zero-Trust con Tailscale en pfSense 23
5.3. Nodo Tanker: SSL, DNS y Nginx 24
5.4. FoxEngine: APK nativa y red Mesh 25
5.5. Plant: BorgBackup + interfaz web 25
6. Demostración, repositorio y conclusiones 27
Demo en vídeo 27
Repositorio en GitHub 27
Conclusiones 27

1. Introducción y contexto del proyecto

1.1. Idea y motivación

La idea detrás de **Fox Engine** no nació como un proyecto académico al uso, sino como una pregunta práctica: ¿hasta dónde podría llegar un administrador de sistemas si tuviera un "ayudante" que viviera dentro de su propia infraestructura, conociera la topología de red y pudiera ejecutar comandos sobre los servidores en cuestión de segundos desde el móvil? Esa es la motivación que ha guiado todo el trabajo.

A partir de ahí, el proyecto crece en dos direcciones que conviven: por un lado, montamos un **entorno empresarial real** (firewall perimetral, directorio centralizado, orquestación con Docker Swarm, monitorización, copias de seguridad, helpdesk, gestor de contraseñas, etc.), y por otro le ponemos encima un **cerebro de IA** capaz de interactuar con ese entorno bajo supervisión humana. La gracia está en cómo encajan las dos piezas: no es "una IA por un lado y unos servidores por otro", sino que la IA conoce a los servidores y los servidores aceptan órdenes de la IA.

Como remate, hemos preparado un escenario de **"Ingeniería del Caos"** para la presentación: provocamos un fallo controlado (saturar disco o tumbar un servicio) delante del tribunal, y la idea es ver cómo Grafana se pone en rojo, el bot avisa por Telegram/APK y, tras la confirmación del operador, lanza el comando que devuelve el sistema a la normalidad.

1.2. El equipo: Unidad Fox Hound

El grupo de trabajo se ha bautizado como Fox Hound, manteniendo la línea temática del proyecto. El reparto de responsabilidades es el siguiente:

Miembro	Responsabilidad principal
Marcos Bolívar Muñoz	Coordinación general del proyecto, Mother Base (LDAP, Docker Swarm, monitorización) y diseño inicial de Fox Engine.
Sebastián Carrillo Medina	Nodo Plant (Passbolt, FreeScout, BorgBackup) y soporte en Mother Base.
Francisco Castillo Martín	Seguridad transversal del proyecto: pfSense, reglas de firewall y configuración de Tailscale.
Javier Pérez Martín	Nodo Tanker: Nginx como proxy inverso, MkDocs, Homepage, Poste y certificados SSL.
Mario Romera Braojos	Desarrollo del cerebro de IA en Fox Engine: migración a Ollama, orquestador multihilo, APK nativa.

1.3. Finalidad y alcance

El objetivo del proyecto es construir un entorno cliente-servidor seguro y funcional, en el que cada pieza tenga sentido por sí misma pero también dentro del conjunto. Concretamente, el proyecto pretende demostrar tres cosas:

- **Que se puede orquestar un clúster real** con tecnologías estándar (Docker Swarm, Prometheus/Grafana, OpenLDAP, pfSense) sobre Proxmox, sin depender de servicios en la nube.
- **Que la IA puede asistir al administrador** sin convertirse en un punto único de fallo: el modelo se ejecuta en local con Ollama, no toca nada sin confirmación humana y se integra de forma controlada en el flujo de trabajo.
- **Que el sistema es auditable y resistente**: hay perímetro, hay segmentación, hay backups cifrados, hay identidades centralizadas y todo deja huella en logs o métricas.2. Visión global de la arquitectura

2. Visión global de la arquitectura

Antes de entrar en los detalles de cada máquina, conviene entender el proyecto como un conjunto. Esta sección presenta **cómo está montado todo**, qué hace cada nodo y, sobre todo, qué papel cumple dentro del ecosistema. En la siguiente sección (apartado 3) se explican los flujos de comunicación concretos entre piezas, que es donde se ve realmente cómo dialogan los sistemas entre sí.

2.1. Esquema general de funcionamiento

La arquitectura está dividida en dos zonas claramente diferenciadas:

- **Zona externa (WAN):** todo lo que queda fuera de nuestra red — Internet, los dispositivos móviles del equipo, los servidores de Tailscale para la red Mesh y, en el centro educativo, la red interna del instituto. A esta zona no le exponemos puertos críticos; lo único que hay "cara a internet" es pfSense.
- **Zona interna (LAN 192.168.1.0/24):** una red privada y aislada que vive detrás del firewall. Aquí residen los cuatro nodos del proyecto (Mother Base, Fox Engine, Plant y Tanker), y aquí ocurre toda la operación interna: monitorización, autenticación LDAP, despliegues con Docker Swarm, copias de seguridad y servicios web.

El siguiente diagrama resume cómo encajan las piezas y por dónde circula el tráfico:

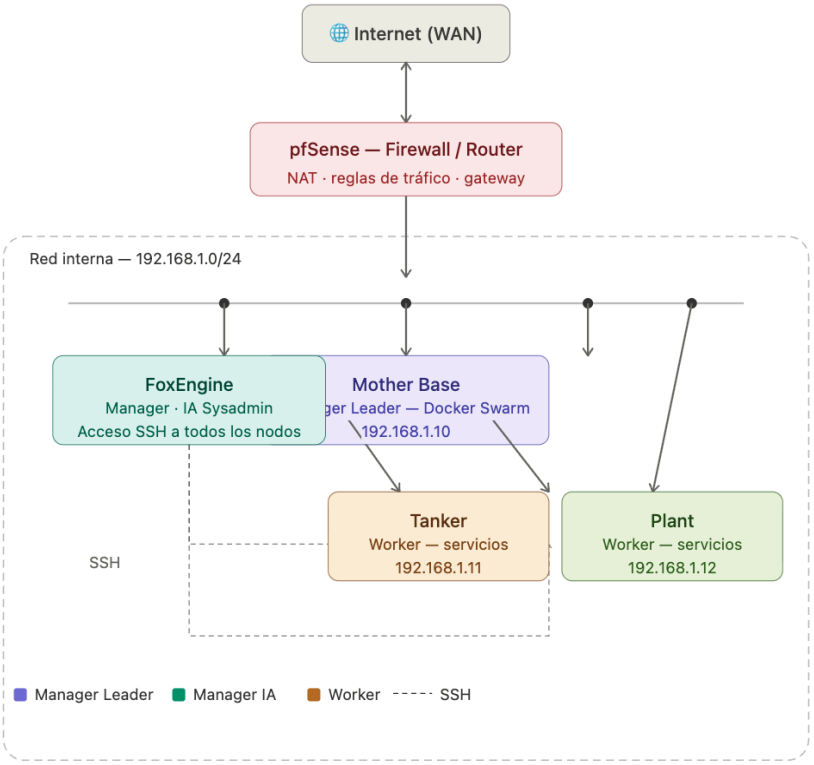

Figura 1. Esquema general: pfSense como única puerta de entrada, Mother Base como Manager Leader del Swarm, Fox Engine con acceso SSH a todos los nodos (líneas discontinuas) y los workers Tanker y Plant.

Leyendo el diagrama de arriba hacia abajo: el tráfico externo entra siempre por pfSense, que actúa como router, firewall y gateway. Detrás de pfSense, en la red 192.168.1.0/24, vive el clúster. Mother Base es el Manager Leader del Docker Swarm (dirige los despliegues y aloja la pila de monitorización); Tanker y Plant son los workers donde se levantan los servicios; y Fox Engine, aunque también es manager del Swarm, está separado conceptualmente porque su misión es distinta: ejecutar la IA y mandar órdenes vía SSH a los demás nodos cuando el operador lo autoriza.

2.2. Inventario de nodos y su papel

Cada máquina tiene un nombre, un dueño y una función bien definida. La tabla siguiente resume el reparto:

Nodo	Función dentro del ecosistema
pfSense (El Guardián)	Perímetro de la red. Hace de router/firewall entre la WAN y la LAN, gestiona DHCP estático para los nodos y aloja el cliente Tailscale para el acceso remoto Zero-Trust.
Mother Base (192.168.1.10)	Centro neurálgico. Aloja OpenLDAP (identidades), DNS interno, Prometheus + Grafana (monitorización) y actúa como Manager Leader del Docker Swarm.
Fox Engine	Cerebro de IA. Ejecuta Ollama (qwen2.5-coder) en local y el orquestador Python (Flask + Telebot) que recibe webhooks y comandos del operador. Tiene acceso SSH sin contraseña a los demás nodos para ejecutar las órdenes confirmadas.
Plant (192.168.1.12)	Worker del Swarm. Aloja servicios de soporte para los administradores: Passbolt (contraseñas), FreeScout (helpdesk) y BorgBackup con interfaz web (copias).
Tanker (192.168.1.11)	Worker del Swarm. Es el frontend/punto de unión del clúster: Nginx como proxy inverso con SSL, MkDocs para documentación técnica, Homepage centraliza el acceso a todos los servicios y Poste gestiona mensajería y permite logins con correos privados. Poste.io

2.3. Recursos asignados en Proxmox

Todo el clúster está virtualizado en el Proxmox del centro. Cada nodo se ha dimensionado en función del trabajo que tiene que hacer: Mother Base necesita más memoria por la pila de monitorización; pfSense apenas necesita nada porque solo hace rutado.

Nodo	SO	vCPU / RAM / Disco
pfSense	pfSense CE	1 / 1 GB / 8 GB
Mother Base	Debian 12	2 / 4 GB / 30 GB
Fox Engine	Debian 12	9 / 32 GB / 30 GB
Tanker	Debian 12	2 / 2 GB / 20 GB
Plant	Debian 12	2 / 2 GB / 20 GB

> pfSense lleva dos tarjetas de red virtuales: una conectada al bridge del instituto (vmbr0, WAN) y otra a un bridge interno aislado (vmbr1, LAN). Esto es lo que permite que el resto de nodos viva en una red 192.168.1.0/24 propia sin pisar la red del centro y, lo más importante, que podamos simular ataques sin riesgo para la infraestructura externa.

3. Comunicaciones e interdependencias entre sistemas

Esta es la sección clave para entender **cómo dialogan las piezas del proyecto**: qué servidor habla con qué servidor, qué protocolo usan, qué hace falta que esté operativo para que otra cosa funcione, y dónde están los puntos de fallo. La hemos organizado en cuatro flujos representativos —los que más se usan en el día a día del proyecto— y un mapa final de dependencias.

3.1. Flujo 1 — Petición remota: del móvil al servidor ejecutado

Este es el flujo "estrella" del proyecto y el que más sistemas atraviesa. Imaginemos que un administrador del equipo quiere consultar el estado de Mother Base desde su móvil:

1. El administrador escribe en la APK (o en Telegram) un comando como *"/haz me puedes decir el nombre de mi máquina"*, habiendo seleccionado previamente "MotherBase" como objetivo.

2. El mensaje viaja a través de la red Tailscale (basada en WireGuard) hasta el contenedor del bot Python que vive en Fox Engine. La conexión es punto a punto y cifrada; el servidor no expone puertos a Internet.

3. El bot valida que el CHAT_ID o el token de la APK pertenezca a alguien autorizado, y consulta a Ollama (qwen2.5-coder) en local para traducir la pregunta natural a un comando Bash: hostname.

4. El bot envía al usuario una propuesta de comando con dos botones: ✅ EJECUTAR y ❌ CANCELAR. Hasta aquí no se ha tocado nada en ningún servidor.

5. Al pulsar EJECUTAR, Fox Engine abre una conexión SSH (con clave ED25519, sin contraseña) contra Mother Base, ejecuta hostname y devuelve la salida al usuario.

6. La respuesta vuelve por el mismo camino: SSH ➜ bot Python ➜ Tailscale ➜ APK.

Figura 2. Vista del flujo desde la APK: pregunta en lenguaje natural ➜ comando propuesto ➜ confirmación humana ➜ ejecución remota ➜ resultado de hostname.

Lo importante de este flujo es que **en ningún momento la IA toca un servidor sin autorización**. Si el operador no pulsa EJECUTAR, el comando no sale del bot. Además, todas las piezas necesarias (red Tailscale, bot, Ollama, claves SSH) deben estar arriba para que funcione; si una falla, el flujo se detiene en ese punto.

3.2. Flujo 2 — Identidad centralizada: login con LDAP

Una de las decisiones de diseño del proyecto fue no gestionar usuarios locales en cada servidor. En lugar de eso, todas las cuentas viven en un OpenLDAP central que corre en Mother Base. Cuando cualquier miembro del equipo se conecta a un nodo, el proceso es:

1. El usuario hace SSH (o se loguea por consola) contra, por ejemplo, Tanker.

2. PAM en Tanker no encuentra el usuario en /etc/passwd, así que consulta al servicio libnss-ldapd, que está configurado para apuntar al servidor LDAP en 192.168.1.10:389.

3. OpenLDAP en Mother Base valida la credencial contra su base de datos (dominio dc=tfg,dc=local) y devuelve el perfil del usuario.

4. PAM en Tanker llama al módulo pam_mkhomedir, que crea automáticamente /home/usuario si es la primera vez que entra en esa máquina.

5. El usuario tiene shell y home en Tanker, sin que ningún administrador haya tenido que crear la cuenta a mano allí. Si se da de alta a un nuevo compañero en phpLDAPadmin (que también vive en Mother Base), pasa a tener acceso a los tres nodos sin tocar nada más.

> Interdependencia importante: si Mother Base se cae, los usuarios nuevos no pueden loguearse en Tanker ni en Plant. Por eso las claves SSH ED25519 son la red de seguridad — el equipo de operaciones siempre puede entrar aunque LDAP esté caído.

3.3. Flujo 3 — Monitorización y autocuración

La monitorización es bidireccional: Prometheus recoge datos de las máquinas y, cuando algo se sale de madre, dispara una alerta hacia Fox Engine para que reaccione (o más bien, para que proponga al operador una reacción). El recorrido es:

1. En cada nodo (Mother Base, Tanker, Plant, Fox Engine) corre un contenedor node-exporter que expone métricas del sistema en el puerto 9100.

2. Prometheus, que vive en Mother Base, hace scrape cada 15 segundos de todos esos exporters. La configuración usa DNS dinámico del Swarm (dns_sd_configs sobre tasks.node-exporter), de modo que si añadimos un nodo nuevo al clúster, Prometheus lo descubre solo.

3. Las métricas se visualizan en Grafana (también en Mother Base, puerto 3000) con un dashboard tipo "Node Exporter Full" que permite filtrar por instancia y ver CPU, RAM, disco y red de cualquier nodo.

4. Cuando una métrica supera un umbral, Alertmanager lanza un webhook hacia el endpoint Flask que escucha en el puerto 5000 de Fox Engine.

5. El orquestador Python recibe el webhook, consulta a Ollama una posible solución y, en lugar de ejecutarla directamente, manda al operador un mensaje del estilo "Alerta: disco al 95% en Plant. Comando propuesto: docker system prune -af. ¿Ejecutar?".

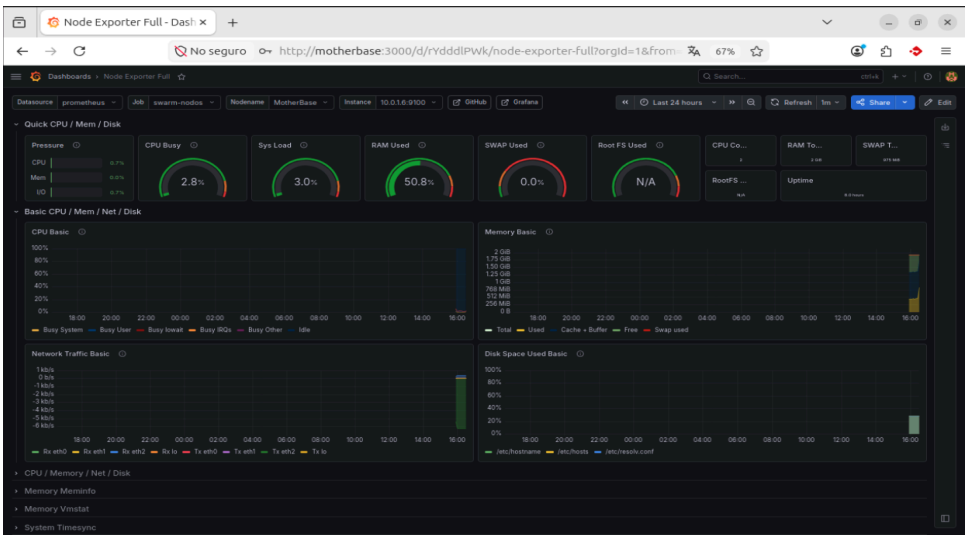

Figura 3. Dashboard de Grafana mostrando las métricas que Prometheus recoge de los node-exporter desplegados en el clúster.

3.4. Flujo 4 — Acceso unificado: DNS interno + Nginx

Para que el equipo no tenga que recordar IPs y puertos (3000 para Grafana, 8080 para FreeScout, etc.), hemos montado una capa de resolución de nombres y enrutamiento que funciona así:

1. Mother Base aloja un servidor DNS interno (un contenedor sencillo en Docker) que resuelve los nombres del dominio *.engine.fox a las IPs internas correspondientes.

2. pfSense entrega esa IP de Mother Base como servidor DNS principal por DHCP, de modo que cualquier máquina conectada a la LAN pueda resolver wiki.engine.fox, grafana.engine.fox, mail.engine.fox, etc.

3. El equipo apunta su navegador a, por ejemplo, https://wiki.engine.fox. La petición llega a Tanker (que es donde está Nginx).

4. Nginx, que actúa como proxy inverso, mira el server_name y redirige la petición al contenedor correcto (MkDocs en este caso) por la red interna de Docker, sin que el contenedor exponga puertos al sistema base.

5. Nginx además presenta el tráfico cifrado con SSL gracias a un certificado autofirmado emitido por nuestra propia CA (FOX Engine Root CA).

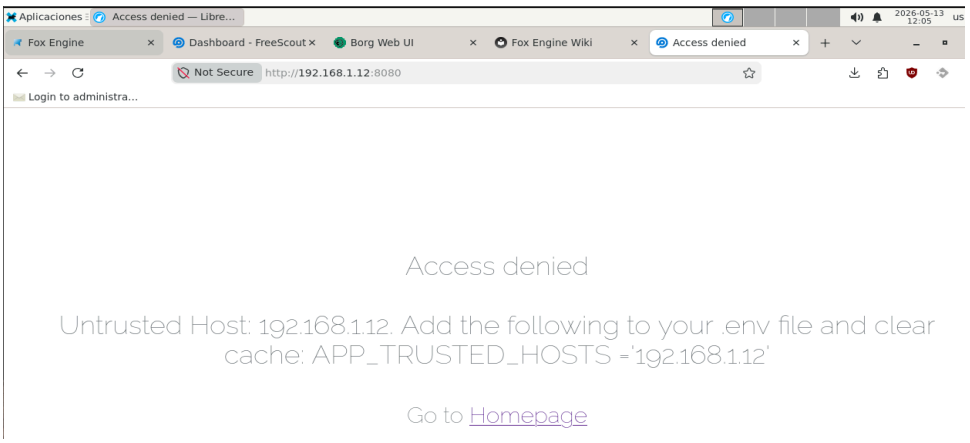

Figura 4. Pestañas abiertas simultáneamente en el navegador: Fox Engine, FreeScout, Borg Web UI y Fox Engine Wiki, todas resolviéndose por DNS interno y servidas por Nginx en Tanker.

3.5. Mapa de dependencias entre servicios

Para que quede claro qué necesita estar arriba para que otra cosa funcione, este es el mapa de dependencias del proyecto. Se lee así: "el servicio de la izquierda depende del servicio de la derecha":

Servicio	Depende de	Consecuencia si la dependencia cae
Login en Tanker / Plant	OpenLDAP (Mother Base) + red LAN	Solo entran usuarios con clave SSH preconfigurada.
Resolución *.engine.fox	DNS contenedor (Mother Base) + pfSense	Hay que acceder a los servicios por IP:puerto.
Webs publicadas en wiki/mail/grafana	Nginx (Tanker) + DNS	Los contenedores siguen funcionando, pero solo accesibles directamente.
Despliegue con Docker Swarm	Mother Base (Manager Leader)	Plant y Tanker siguen ejecutando lo que ya tenían, pero no admiten redeployments.
Alertas automatizadas	Prometheus + Alertmanager + Fox Engine	Métricas siguen mostrándose en Grafana, pero no hay reacción de la IA.
Comando vía APK/Telegram	Tailscale + Ollama + SSH ED25519	No se pueden ejecutar acciones remotas; sí se puede operar localmente.
Acceso remoto al clúster	pfSense + Tailscale	Solo acceso físico o desde la red del centro.
Copias de seguridad	BorgBackup (Plant) + repositorio AWS S3	Las copias incrementales internas siguen, pero no se replican a la nube.

La lectura general de la tabla es que **Mother Base es el punto más crítico** (LDAP, DNS y Swarm Manager), seguido de pfSense (perímetro y rutado interno). El resto de servicios degradan con elegancia: si Fox Engine se cae, la infraestructura sigue funcionando perfectamente sin IA; si Plant se cae, los administradores pierden las herramientas de soporte pero el frontend de Tanker sigue sirviendo páginas.

4. Implementación por nodo

Una vez explicada la visión global y los flujos de comunicación, en esta sección bajamos al detalle técnico de cada nodo: cómo está montado, qué decisiones de configuración hemos tomado y por qué. Cada apartado termina con un párrafo titulado "Cómo se integra con el resto", para enlazar siempre con la visión de conjunto.

4.1. pfSense — El Guardián del perímetro

pfSense es la primera línea del proyecto. Es la única máquina que tiene contacto con la red exterior y, por tanto, la responsable de que el resto del clúster pueda vivir en una red privada y aislada. Hemos elegido la edición **pfSense CE** porque es estable, gratuita y tiene un repositorio de paquetes bastante grande (entre ellos el cliente Tailscale, que más adelante usamos para el acceso remoto).

Configuración de red e interfaces

La máquina tiene dos NIC virtuales:

- WAN: conectada al bridge del instituto, recibe IP por DHCP de la red del centro.
- LAN: conectada a un bridge interno de Proxmox (sin acceso al mundo exterior salvo a través de pfSense), con IP 192.168.1.1/24.

En el servidor DHCP de la LAN hemos definido un rango (192.168.1.100 — 192.168.1.245) y, además, hemos creado reservas estáticas para los tres nodos principales para que siempre cojan la misma IP:

DHCP Static Mappings for this Interface (total: 3)

Static ARP	MAC address	Client Id	IP address	Hostname	Description	
	bc:24:11:2b:e4:3e	motherbase	192.168.1.10	motherbase	motherbase	
	bc:24:11:8e:00:b1	tanker	192.168.1.11	tanker	tanker	
	bc:24:11:12:9e:a4	plant	192.168.1.12	plant	plant	

Figura 5. Tabla de DHCP Static Mappings de pfSense: motherbase (.10), tanker (.11) y plant (.12).

Reglas de firewall y NAT

Las reglas de pfSense siguen una filosofía de "denegar todo, permitir lo necesario". Lo más relevante:

- Por defecto, ningún puerto está abierto desde la WAN hacia los nodos internos. El único tráfico entrante que se permite es el de la red Tailscale, que entra cifrado por UDP/41641.

- Desde la LAN se permite salida a Internet con NAT estándar, para que los nodos puedan actualizar paquetes, descargar imágenes Docker, etc.
- Existe una regla específica que permite a Fox Engine alcanzar los puertos SSH (22) de Mother Base, Tanker y Plant; el resto de comunicaciones SSH están restringidas al rango del clúster.

Cómo se integra con el resto

pfSense es la base sobre la que se apoya todo. Sin él no hay red interna, no hay DHCP, no hay DNS distribuido y no hay manera de que el resto del equipo se conecte al clúster desde fuera. Su disponibilidad es crítica: si pfSense cae, el clúster queda incomunicado.

4.2. Mother Base — Cerebro de identidades y monitor

Mother Base es el centro neurálgico de la operación. En esta máquina conviven cuatro grandes responsabilidades: identidades (OpenLDAP), resolución de nombres (DNS interno), orquestación (Manager Leader del Swarm) y monitorización (Prometheus + Grafana).

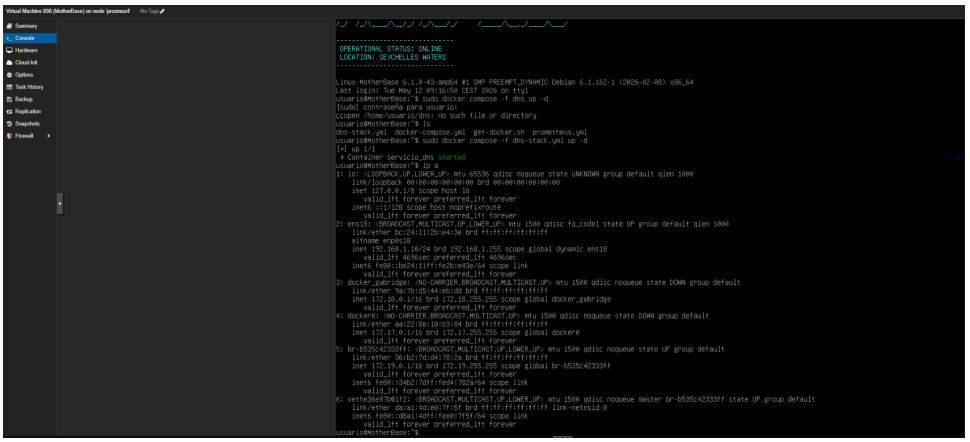

Figura 6. Consola de Mother Base: levantamiento del stack DNS y verificación de interfaces (ens18 = 192.168.1.10, bridges de Docker).

OpenLDAP y phpLDAPadmin

Se ha desplegado un **servicio de directorio centralizado** con OpenLDAP, sirviendo el dominio dc=tfg,dc=local. La gestión de unidades organizativas (ou=people, ou=groups) se hace a través de phpLDAPadmin, una interfaz web que también vive en Mother Base. Esto convierte a Mother Base en la autoridad de identidad del proyecto: cualquier alta o baja de usuario se hace aquí y se propaga automáticamente.

Docker Swarm — Manager Leader

Sobre Mother Base corre el plano de control del Swarm. Es el nodo donde se ejecutan los docker stack deploy y donde se reciben las suscripciones de los workers (Tanker y Plant). El comando que arranca la pila de monitorización es:

```
docker stack deploy -c docker-compose.yml monitor
```

Y el fichero docker-compose.yml de la pila de monitorización es este:

```
services:
  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml"]
    deploy:
      placement:
        constraints: [node.role==manager]
  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
    deploy:
      placement:
        constraints: [node.role==manager]
  node-exporter:
    image: prom/node-exporter
    hostname: '{{.Node.Hostname}}'
    deploy:
      mode: global
```

Las restricciones de despliegue son importantes: Prometheus y Grafana solo se levantan en nodos manager (es decir, en Mother Base), mientras que node-exporter se despliega en mode: global, lo que significa que se ejecuta una instancia en cada nodo del Swarm. Así Prometheus puede hacerle scrape a todos a la vez.

El fichero prometheus.yml correspondiente es minimalista pero funcional:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'swarm'
    dns_sd_configs:
      - names: ['tasks.node-exporter']
        type: 'A'
        port: 9100
```

DNS interno

Mother Base aloja también un servidor DNS sencillo dentro de un contenedor Docker, que resuelve el dominio *.engine.fox a las IPs internas. pfSense entrega esta IP (192.168.1.10) como DNS principal vía DHCP, de modo que toda la LAN puede usar nombres en lugar de IPs.

Grafana — el "Radar"

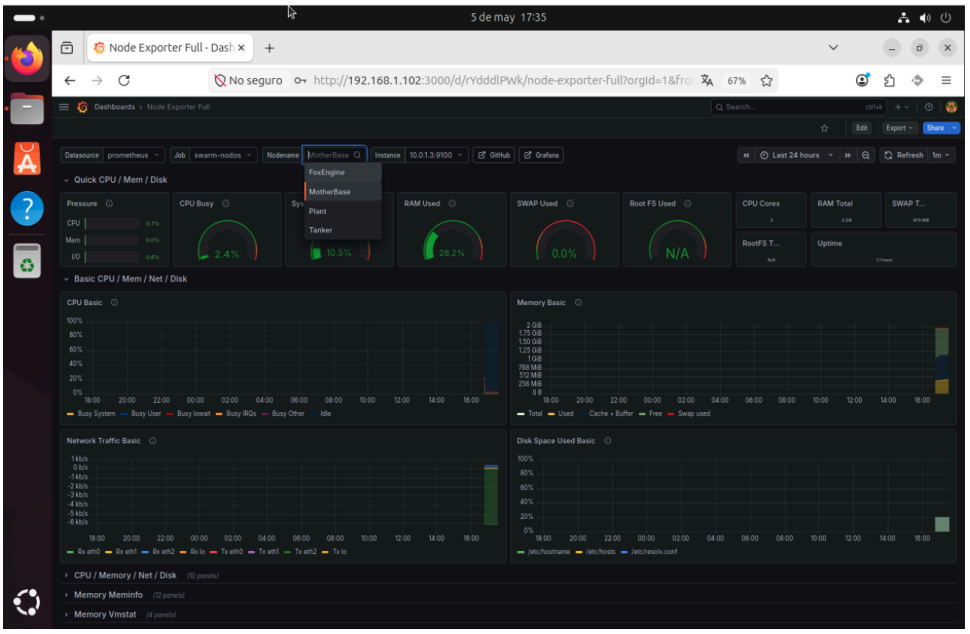

Figura 7. Dashboard de Grafana con el selector de Nodename desplegado: FoxEngine, MotherBase, Plant y Tanker disponibles para inspeccionar.

El dashboard que usamos es una adaptación del "Node Exporter Full". Lo importante de esta vista es que el selector de instancia permite cambiar entre nodos en caliente, y que las gráficas reaccionan en tiempo real ante un evento (esto es precisamente lo que hace tan visual la demo de Ingeniería del Caos).

Cómo se integra con el resto

Mother Base es la pieza central del proyecto y la que más servicios "consume" de y "ofrece" al resto. Provee identidades a Tanker y Plant, recibe métricas de todos los nodos vía node-exporter, le indica a pfSense quién es el DNS interno y, cuando Alertmanager detecta una anomalía, dispara un webhook hacia Fox Engine. Si esta máquina cae, la infraestructura sigue funcionando pero deja de tener gestión centralizada.

4.3. Fox Engine — El cerebro de IA

Fox Engine es la pieza más singular del proyecto. Su misión es ejecutar el modelo de IA, recibir órdenes del operador (sea por Telegram, sea por la APK propia) y, una vez confirmada la acción, traducirla en una ejecución real sobre los servidores del clúster.

Migración a inferencia local: Ollama

En la fase inicial el bot consultaba a Google Gemini por API. Para la entrega final hemos hecho una **migración completa a inferencia local** usando Ollama, con el modelo qwen2.5-coder. La decisión responde a tres razones:

- **Privacidad de datos:** la topología de red, credenciales y comandos no salen del servidor.
- **Soberanía tecnológica:** eliminamos la dependencia de cuotas de tokens y de la latencia de APIs comerciales.
- **Especialización:** qwen2.5-coder está afinado para generar Bash y administrar sistemas Linux, que es justo el caso de uso.

Orquestador Python multihilo

El backend está escrito en Python y usa multithreading para gestionar dos flujos en paralelo:

- **Servidor de alertas (Flask).** Escucha en el puerto 5000 esperando webhooks de Alertmanager. Cuando llega una alerta, llama a Ollama para que proponga una solución y se la manda al operador para que confirme.
- **Gestor de mensajería (Telebot + API de la APK).** Un hilo independiente atiende los mensajes que llegan desde el cliente (Telegram o APK propia), procesa el lenguaje natural, prepara el comando y lo somete a confirmación.

Tres capas de seguridad

Dado que el bot tiene permisos para ejecutar comandos en máquinas reales, hemos puesto tres capas de protección encima:

1. Filtrado por identificador único (whitelist). La función es_usuario_autorizado() valida cada interacción contra una lista de CHAT_ID o tokens autorizados. Cualquier intento desde una cuenta desconocida queda bloqueado y registrado.

2. Human-in-the-Loop. Cuando la IA genera un comando, el bot no lo ejecuta — se lo presenta al operador con dos botones: EJECUTAR y CANCELAR. Sin pulsación, no hay acción. Esto es lo que evita "alucinaciones" peligrosas del modelo.

3. Sanitización de la salida del modelo. La función clean_command() elimina etiquetas Markdown, bloques de código y todo lo que no sea Bash puro antes de pasar el comando al sistema operativo.

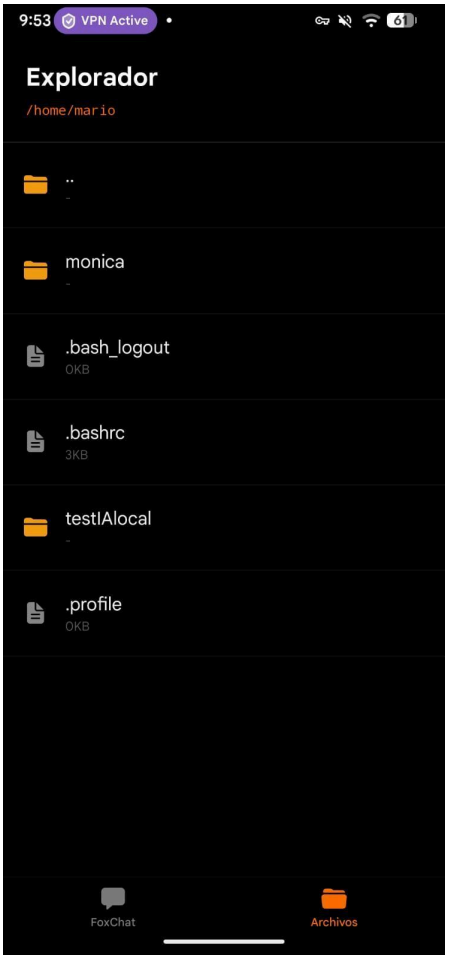

Figura 8. Pestaña "Archivos" de la APK: explorador remoto del nodo seleccionado. Una vez sanitizado y validado el comando, el operador puede además navegar por el sistema de ficheros del servidor sin tener que abrir una sesión SSH manual.

Modos de operación y autocuración

El bot distingue entre conversación y ejecución técnica mediante prefijos:

- **Modo consultivo (texto libre).** El bot actúa como un sysadmin experto y responde en lenguaje natural sin tocar nada en los servidores. Usa el contexto general CONTEXTO_GRAL.
- **Modo ejecutor (/haz <petición>).** Activa un prompt estricto que obliga al modelo a devolver únicamente código Bash. Tras la confirmación, el comando se lanza por subprocess.run con shell=True y captura completa de stdout/stderr, que se devuelve al usuario.
- **Autocuración (Self-healing).** Ante una alerta de Flask, el bot consulta a Ollama una posible solución y se la propone al operador. Si este confirma, se ejecuta. Si no, se queda como un aviso en el chat.

Cliente nativo (APK)

Para no depender en exclusiva de los servidores de Telegram, hemos desarrollado una APK propia que actúa como cliente principal. Aporta tres ventajas:

- **Desacoplamiento:** si Telegram falla, el flujo sigue funcionando por la app propia.

- **Explorador y editor remoto:** la APK incluye un navegador de ficheros y un editor de texto, así el administrador puede modificar configuraciones directamente desde el móvil.
- **Selector multi-nodo:** un desplegable permite cambiar el destino de los comandos sobre la marcha, entre FoxEngine, Plant, MotherBase y Tanker.

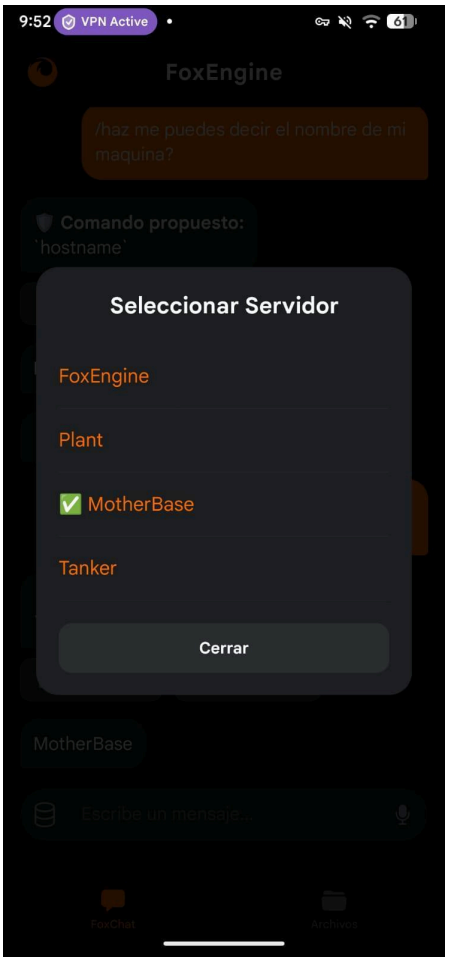

Figura 9. Selector de servidor en la APK: MotherBase seleccionada como objetivo de los siguientes comandos.

Conectividad: Tailscale (Mesh)

La APK no se conecta directamente a Internet — se conecta a la red privada Tailscale. Tailscale crea una red overlay basada en WireGuard que permite que el móvil y Fox Engine se comuniquen como si estuvieran en la misma LAN, sin abrir puertos en pfSense. La seguridad la garantiza la propia capa de red (autenticación + cifrado punto a punto), de modo que la comunicación entre APK y bot puede ir por HTTP plano simplificando enormemente la configuración.

Cómo se integra con el resto

Fox Engine es un manager del Swarm pero, a efectos de operación, su misión es claramente distinta del resto: **orquestar acciones** sobre los demás nodos. Para ello necesita acceso SSH (con claves ED25519) a Mother Base, Tanker y Plant; necesita el endpoint de Alertmanager apuntando a su puerto 5000; y necesita que Tailscale esté operativo para que el equipo pueda alcanzarlo desde fuera. Si Fox Engine se cae, la infraestructura sigue siendo plenamente funcional — simplemente, se pierde la capa de automatización y asistencia.

4.4. Plant — Servicios de soporte

Plant es el worker del Swarm dedicado a alojar las herramientas que los administradores usan en su día a día. La idea es que cualquier servicio "de oficina" del equipo IT (helpdesk, gestor de contraseñas, copias) viva aquí, dejando libre a Tanker para el frontend.

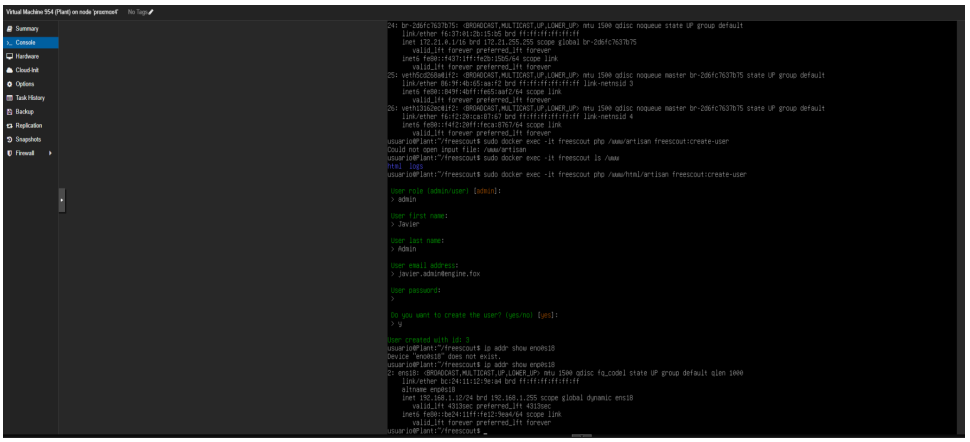

Figura 10. Consola de Plant: creación de un usuario en FreeScout y validación de interfaces (ens18 = 192.168.1.12) con los bridges de Docker.

Passbolt — Gestor de contraseñas

Passbolt es una solución open-source de gestión de contraseñas orientada a equipos técnicos. Su gran baza es que utiliza **OpenPGP** como base de su modelo de seguridad. Esto significa cifrado de extremo a extremo y, sobre todo, que las claves privadas se generan y se quedan en el cliente; el servidor solo guarda claves públicas y secretos cifrados.

Arquitectónicamente, Passbolt sigue un patrón MVC: el backend está en PHP con CakePHP (lógica de negocio, autenticación y API REST), el cliente es una extensión de navegador que integra el motor criptográfico OpenPGP, y los datos se almacenan en una base de datos MySQL/MariaDB.

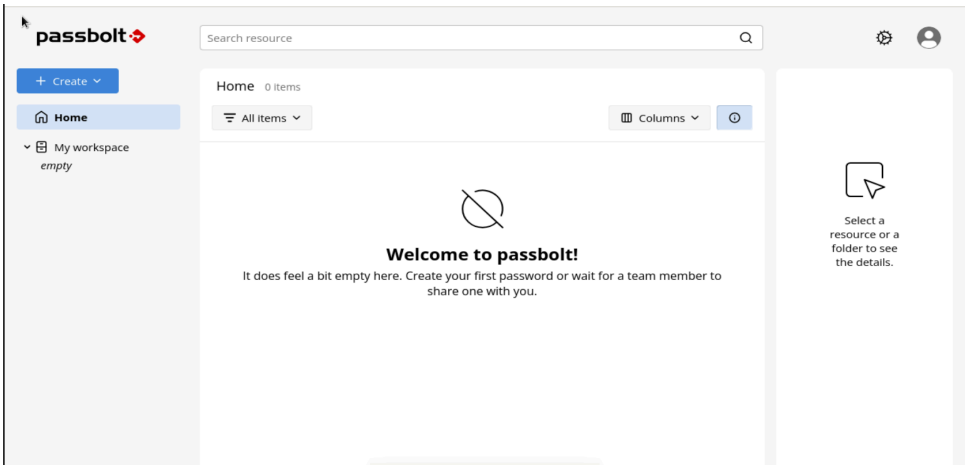

Figura 11. Interfaz de Passbolt recién desplegada en Plant, lista para que el equipo empiece a guardar credenciales compartidas.

FreeScout — Helpdesk

Para gestionar los tickets de soporte interno usamos FreeScout, una alternativa ligera y open-source a Help Scout. Está construida sobre Laravel 5.5 (PHP) y su núcleo es muy simple — incluye exactamente lo que necesita un helpdesk básico — y se extiende mediante módulos opcionales gratuitos.

Lo más interesante de FreeScout es su integración con el correo: se conecta a las bandejas vía IMAP y SMTP, así que los mensajes que entran al buzón soporte@engine.fox se convierten automáticamente en tickets, y las respuestas desde la interfaz salen como correos normales. Esto lo hace compatible con cualquier servidor SMTP (Gmail, Office 365, Postfix propio). Además, activando el módulo "API & Webhooks" se puede integrar con otras herramientas externas vía REST.

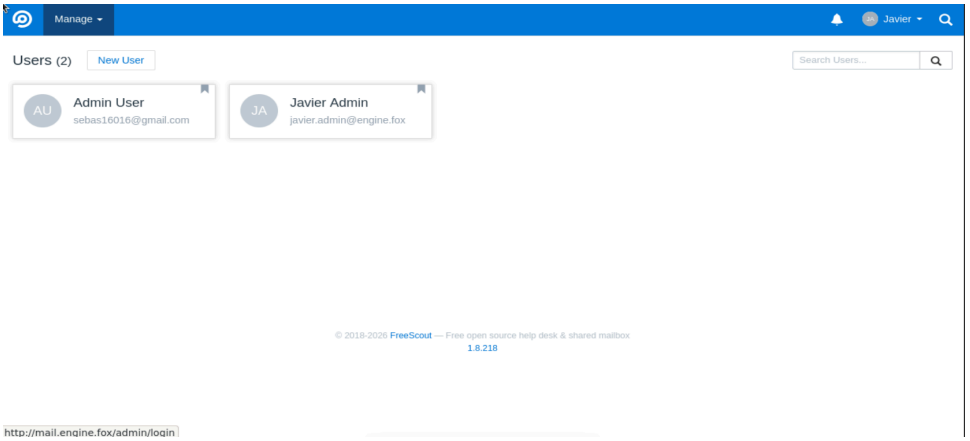

Figura 12. FreeScout corriendo en Plant: alta de los usuarios administradores (admin y Javier Admin) con su dirección de correo en el dominio del proyecto.

Script de backup

Sobre Plant también corre un script diario de copias de seguridad incrementales que replica los directorios críticos de los servidores principales hacia un servidor de respaldo configurado en RAID 1. Cada semana se hace una copia completa, comprimida y cifrada, y para mayor seguridad esta se sube a un bucket de AWS S3.

Cómo se integra con el resto

Plant es worker del Swarm gestionado desde Mother Base. Los usuarios que se loguean en Plant se validan vía LDAP también en Mother Base. Los servicios web que aloja (FreeScout, Passbolt, Borg UI) no se exponen directamente; el tráfico entra por Nginx en Tanker, que hace de proxy inverso. Y la monitorización de Plant llega a Prometheus a través del node-exporter desplegado en modo global.

4.5. Tanker — Frontend y documentación

Tanker es el segundo worker del Swarm, pero a diferencia de Plant tiene un papel más específico: hace de frontend del clúster. Centraliza el tráfico web entrante, presenta un homepage con los enlaces a todos los servicios y aloja la documentación viva del proyecto.

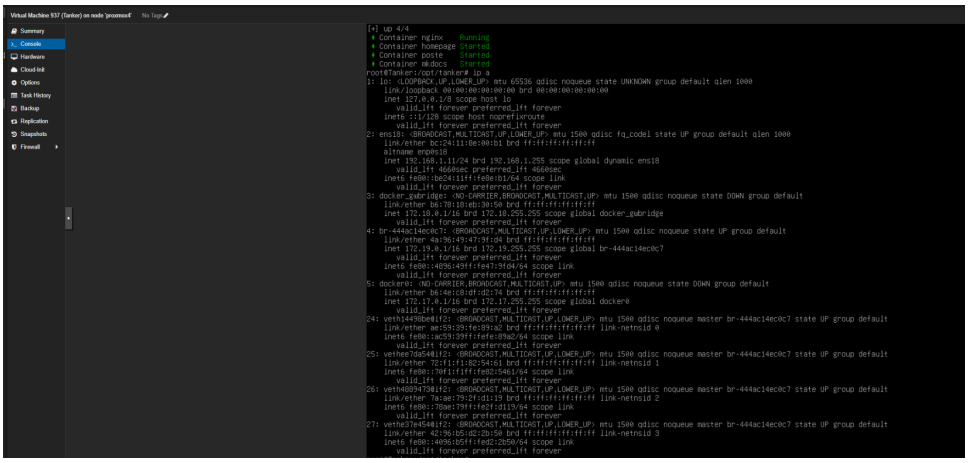

Figura 13. Consola de Tanker: contenedores nginx, homepage, poste y mkdocs en ejecución, e interfaces de red (ens18 = 192.168.1.11).

Nginx como proxy inverso

En lugar de que cada servicio publique su propio puerto al exterior (3000 para Grafana, 8080 para FreeScout, 80 para wiki, etc.), centralizamos toda la entrada en un único contenedor Nginx. La justificación es:

- **Abstracción de puertos:** los administradores acceden a wiki.engine.fox o mail.engine.fox, sin tener que recordar puertos.
- **Aislamiento de red:** al trabajar dentro de una red interna de Docker, los demás contenedores no exponen sus puertos al sistema base. Nginx recibe la petición en 80/443 y la redirige al contenedor correspondiente.
- **Optimización:** Nginx tiene una huella muy pequeña en memoria, algo crítico para un nodo con 2 vCPU y 2 GB de RAM.
- **SSL:** Nginx termina la conexión TLS usando un certificado autofirmado emitido por nuestra CA interna (FOX Engine Root CA), cuyo Common Name es *.engine.fox, lo que permite cubrir todos los subdominios del proyecto con un único certificado.

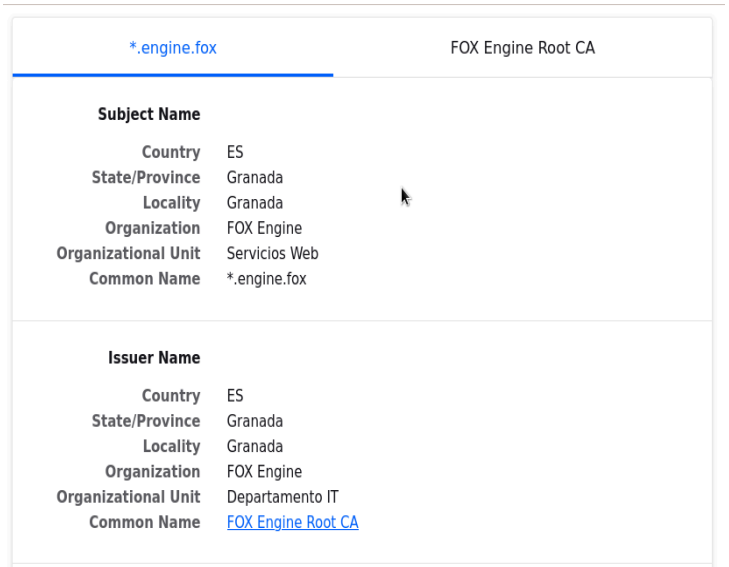

*Figura 14. Detalle del certificado wildcard *.engine.fox emitido por FOX Engine Root CA.*

MkDocs — gestión del conocimiento

Un proyecto de este tamaño necesita documentación que esté a la altura, así que hemos montado MkDocs para tener una wiki técnica que se mantiene en formato Markdown y se publica como sitio estático. Las ventajas son evidentes:

- **Flujo de trabajo colaborativo:** cualquier miembro del equipo puede editar su sección en Markdown. El contenedor de MkDocs regenera el sitio automáticamente.
- **Centralización:** aquí documentamos desde las reglas de pfSense hasta los prompts del modelo qwen2.5-coder. Ante un fallo, cualquier administrador puede abrir Tanker y encontrar el procedimiento.
- **Buscador integrado:** MkDocs genera un índice en tiempo real, así no hay que recorrer carpetas para encontrar una configuración concreta.

Cómo se integra con el resto

Tanker es la cara visible del clúster. Todos los servicios web pasan por aquí, lo que significa que Nginx tiene que conocer el nombre interno de los contenedores que sirven el contenido (mkdocs, freescout, borg-ui...). Para esto se apoya en el DNS interno de Mother Base y en la red overlay del Swarm, que conecta los contenedores entre nodos. Sin Tanker, los servicios siguen vivos pero solo se pueden alcanzar por IP:puerto, lo cual rompe la URL bonita y, sobre todo, el cifrado SSL.

5. Mejoras de la entrega final

Esta sección recoge las novedades específicas que se han incorporado en la última iteración del proyecto, respecto a lo que se entregó en fases anteriores. Cada punto se entiende mejor a la luz de los flujos descritos en el apartado 3.

5.1. Mejoras en Mother Base

- **Servicio DNS interno:** Mother Base ahora resuelve los nombres *.engine.fox para toda la LAN. Esto cierra el flujo 4 descrito anteriormente y permite que pfSense entregue por DHCP la IP del DNS interno.
- **Docker Swarm depurado:** hemos corregido los problemas que impedían a Mother Base salir como Leader del Swarm en el primer arranque (relacionados con la inicialización del cluster y el orden de los advertise-addr).
- **Fox Engine fuera del Swarm "productivo":** hemos sacado a Fox Engine de las restricciones de despliegue de servicios, manteniéndolo como manager pero sin que el orquestador pueda mover contenedores hacia él. Así la IA queda aislada del resto de servicios y no se ve afectada por un fallo de despliegue masivo.
- **Grafana con Prometheus como data source:** el panel de Grafana ahora se alimenta directamente de Prometheus, con un dashboard que ya viene preconfigurado con todos los nodos del clúster.

5.2. Acceso remoto Zero-Trust con Tailscale en pfSense

Hemos sustituido la VPN tradicional (OpenVPN SSL/TLS) por **Tailscale**, una red overlay basada en WireGuard. La razón es práctica: en el centro educativo no se nos permitía abrir puertos entrantes en el cortafuegos perimetral del instituto, lo que hacía inviable una VPN tradicional. Tailscale, sin embargo, usa técnicas de NAT Traversal y establece conexiones salientes desde pfSense hacia los servidores de coordinación, de modo que no necesita ningún puerto abierto desde fuera.

El resultado es un modelo Zero-Trust: cada miembro autenticado del equipo (Marcos, Javi, Sebas, Mario, Paco) tiene su dispositivo registrado en el tailnet y puede alcanzar la red interna 192.168.1.0/24 como si estuviera presencialmente conectado a ella.

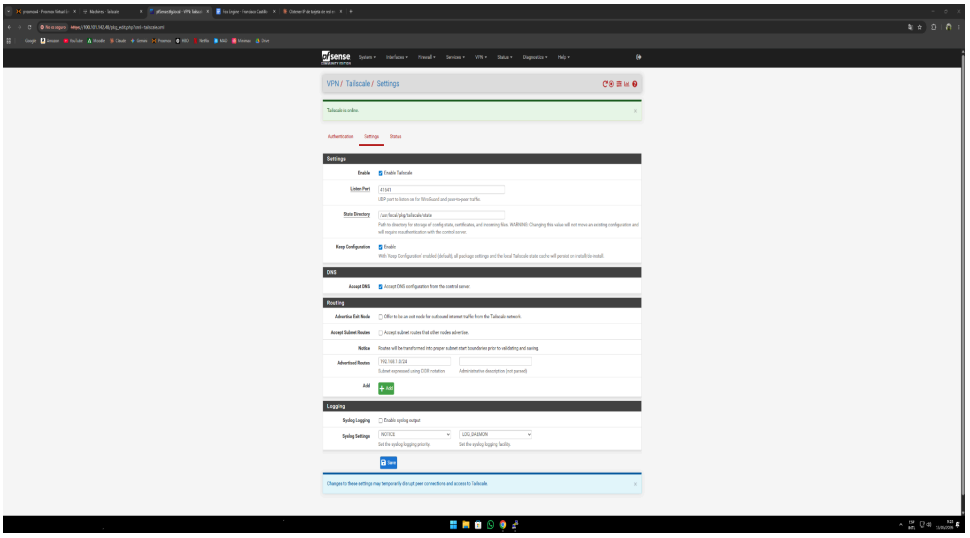

Figura 16. Panel del paquete VPN / Tailscale en pfSense, mostrando Settings, DNS, Routing y Logging.

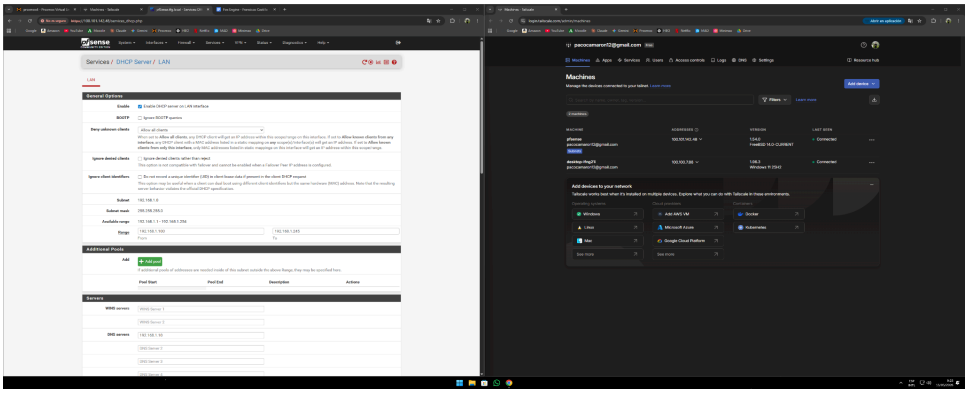

Figura 17. A la izquierda, configuración DHCP en LAN; a la derecha, panel de Tailscale mostrando las máquinas conectadas al tailnet del equipo.

5.3. Nodo Tanker: SSL, DNS y Nginx

Las mejoras en Tanker se centran en reforzar su papel de frontend:

- **Certificado wildcard SSL:** se ha generado *.engine.fox firmado por la CA interna del proyecto, lo que permite cifrar todos los subdominios con un único certificado.
- **Nombres de dominio locales:** los administradores ya no acceden por IP, sino por wiki.engine.fox, mail.engine.fox, borg.engine.fox, etc. — todo gracias al DNS interno de Mother Base.

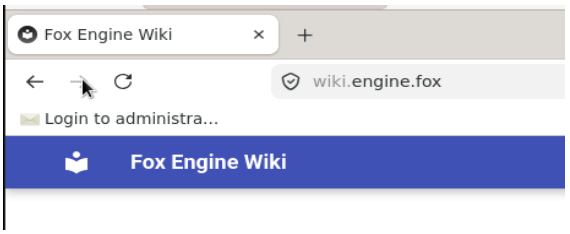

Figura 15. Acceso al sitio MkDocs vía https://wiki.engine.fox, servido por Nginx con certificado wildcard.

- **Aislamiento reforzado:** al colocar a los demás contenedores en una red interna de Docker (sin publicar puertos en el host), una petición directa a IP:puerto desde fuera no llega; o bien Nginx la redirige al servicio correcto, o el contenedor en cuestión rechaza la conexión.

Figura 18. Acceso a Borg UI por su nombre interno y conexión marcada como segura.

- **Portal de Bienvenida Homepage:** Para tener un acceso más visual, sencillo y escalable, hemos añadido el servicio Homepage. Este será el punto de entrada frontend hacia todos los servicios del clúster, evitándose recordar la dirección de cada uno.
    - Integración de Servicios: El panel agrupará de forma visual los accesos a los servicios de cada nodo de forma agrupada y ordenada:

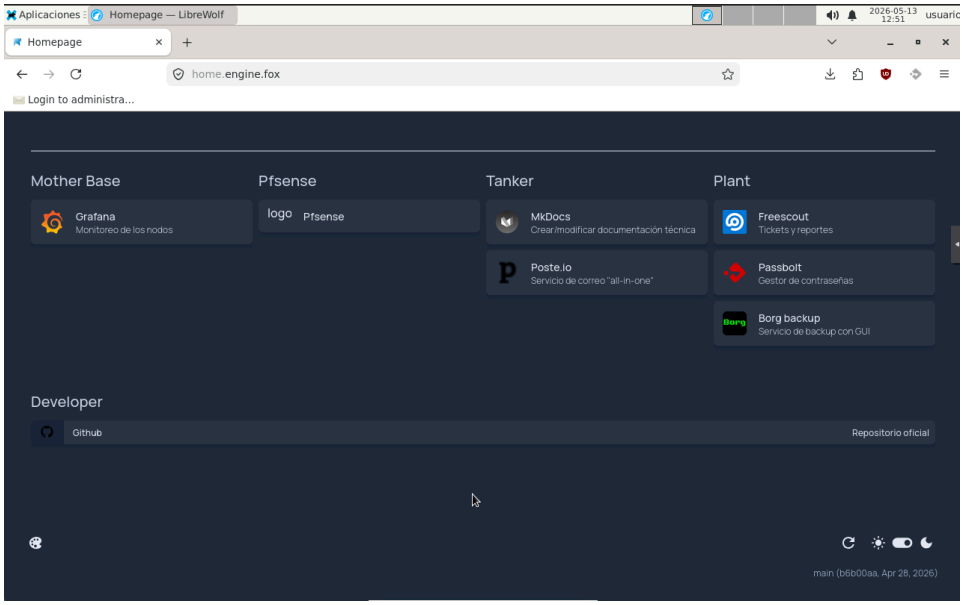

Figura 19. Acceso al portal de bienvenida.

Uso de APIs: El dashboard se configurará para realizar consultas de estado sencillas. Por ejemplo, mediante widgets que revisan el estado de los contenedores/sitios web, y así conocer de primera mano si están accesibles, o sí ha habido algún problema que evite que funcionen:

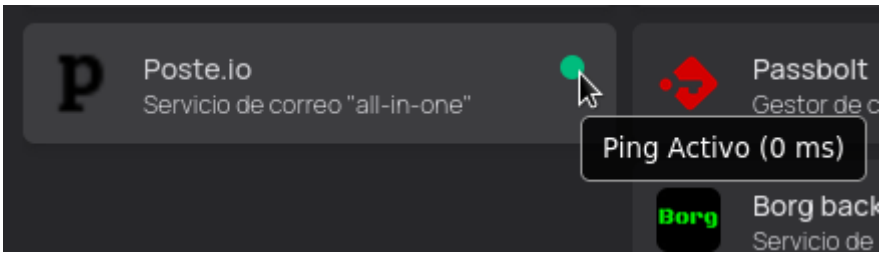

Figura 20. Dashboard de las APIs.

- Debido a que los servicios del nodo Plant tienen sistemas de cuentas con login mediante correo, hemos decidido centralizar, y a su vez, aislar el proceso montando nuestro propio servidor/gestor de correo. La opción más completa era Poste.io.

  Direcciones de correo independientes: al no usar direcciones de correo de Google u otros proveedores, sólo dependemos de nuestro clúster, permitiéndonos incluso usar el servicio sin conexión a internet. También nos permite evitar el uso de correos personales o destinados a otros usos, ya que se accederá a servicios como Passbolt con estos mismos correos.

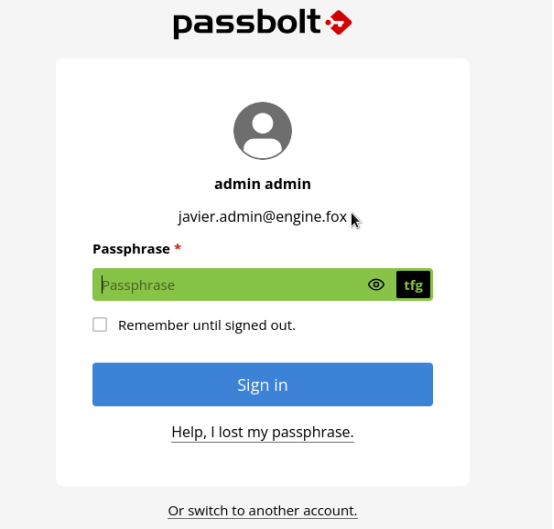

Figura 21. Demo de Correos.

  Mensajería privada: al usar un servidor alojado en una LAN, nadie desde fuera puede interceptar los mensajes o acceder a las cuentas desde internet. Si a eso le sumamos el uso del protocolo SMTP, que cifra los correos, terminamos construyendo una comunicación privada y segura

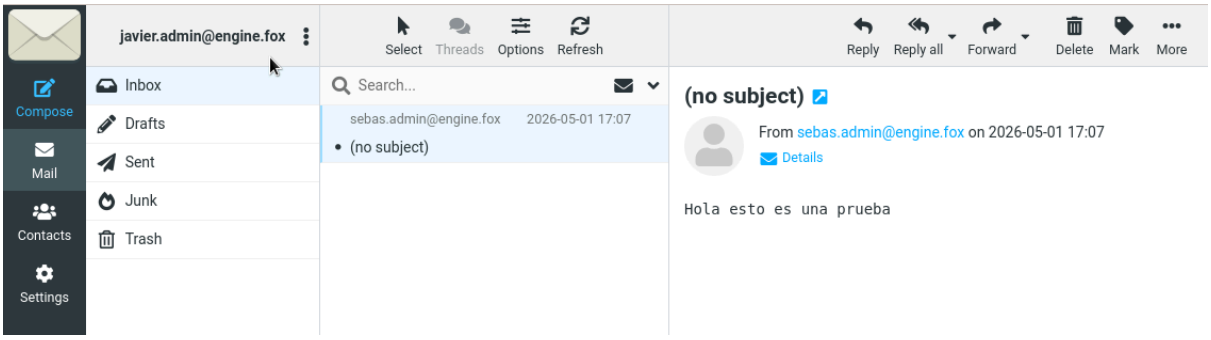

Figura 22. Launcher Poste.Io.

5.4. FoxEngine: APK nativa y red Mesh

Las mejoras más visibles para el usuario final están en Fox Engine, y se resumen en una palabra: **autonomía**. Hemos roto la dependencia de plataformas externas (la nube de Google para la IA, los servidores de Telegram para el chat) sustituyéndolas por componentes propios:

- **APK nativa propia:** aporta selector multi-nodo, explorador de archivos remoto y editor de texto, además de no estar sujeta a las políticas de Telegram.

- **Red Mesh con Tailscale:** la APK y el bot dialogan directamente por la red privada, sin pasar por Internet pública y sin necesidad de SSL/HTTPS adicional (la propia capa de Tailscale ya cifra punto a punto).
- **Topología invisible:** Fox Engine deja de ser un objetivo escaneable desde Internet — desde fuera, simplemente no existe.

5.5. Plant: BorgBackup + interfaz web

Hemos sustituido el script casero de backups por **BorgBackup** con su interfaz web Borg UI 2.0. BorgBackup es la herramienta de copias de referencia en el mundo Linux por tres motivos: deduplicación, cifrado fuerte (AES-256 + HMAC para integridad) y eficiencia.

Lo importante del modelo de seguridad es que el cifrado se hace en el lado del cliente, antes de que los datos salgan del sistema origen. La interfaz web es solo capa de gestión: nunca ve los datos en claro y nunca guarda la clave de cifrado en texto plano. Si alguien comprometiera el servidor de Borg, lo único que encontraría son archivos cifrados sin la clave para abrirlos.

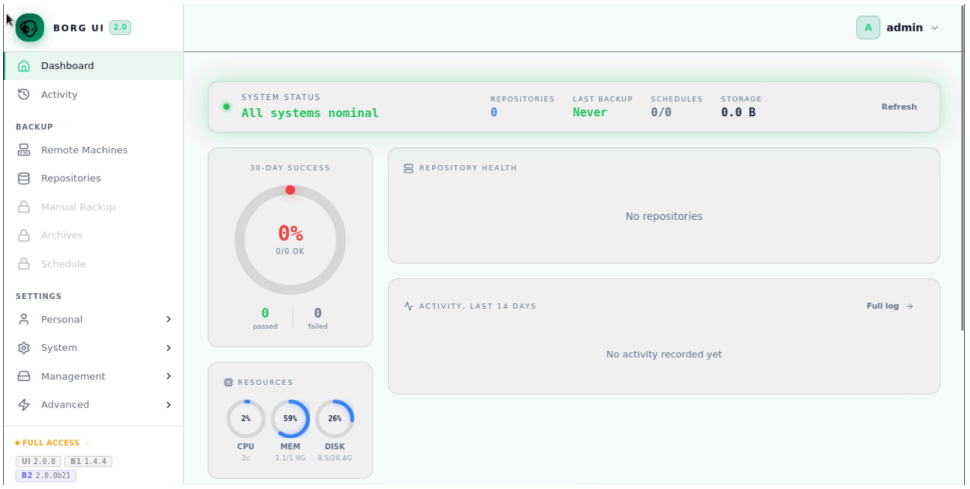

Figura 23. Dashboard de Borg UI 2.0 desplegado en Plant: estado del sistema, resumen de repositorios y uso de recursos.

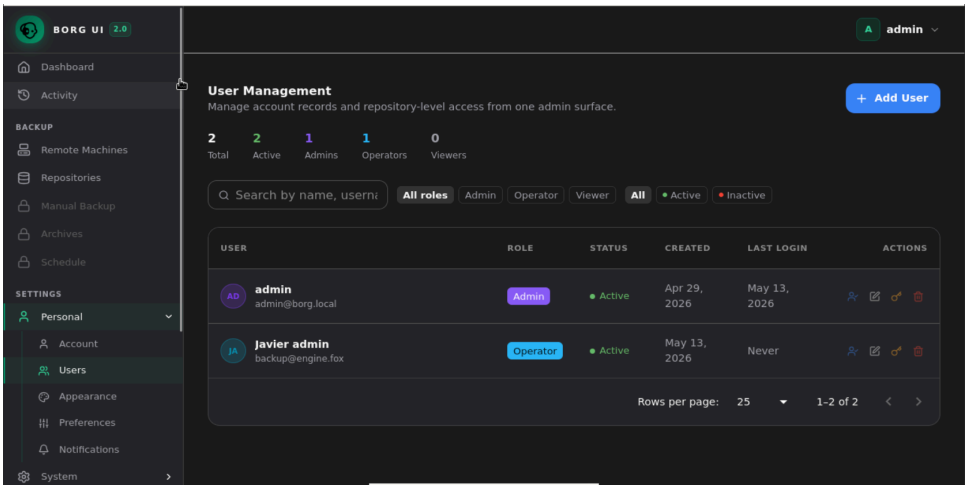

Figura 24. Gestión de usuarios y roles en Borg UI, integrada con el directorio del proyecto (admin y operador Javier).

La arquitectura es cliente-servidor: el núcleo es un servicio Python que vive dentro de un contenedor Docker, ejecuta los comandos de BorgBackup sobre los repositorios montados en el sistema anfitrión y expone una API + UI para gestión. Los repositorios pueden ser locales (en disco RAID 1) o remotos (S3, vía rclone).

6. Demostración, repositorio y conclusiones

Demo en vídeo

Hemos preparado un vídeo corto donde se demuestra el funcionamiento real de la APK y el bot, incluyendo el flujo Human-in-the-Loop y el cambio de objetivo entre nodos. El vídeo está disponible en:

https://youtu.be/C0elM1UrPyM

Repositorio en GitHub

El código del orquestador de Fox Engine y los ficheros de despliegue están publicados en:

https://github.com/MadMark-Fox/FoxEngine

Y específicamente, el código del bot que se ha descrito en el apartado 4.3 se encuentra en:

https://github.com/MadMark-Fox/FoxEngine/blob/main/bot.py

Conclusiones

A lo largo de los meses de trabajo hemos comprobado en primera persona algunas lecciones que merece la pena dejar por escrito.

La primera es que **un entorno empresarial no es la suma de servicios, sino la suma de relaciones entre servicios**. En las primeras entregas teníamos cada nodo perfectamente montado por separado, pero nos faltaba combinarlo: el DNS interno, el flujo de identidades, las dependencias declaradas. Esta entrega final corrige precisamente eso, y por eso hemos puesto la sección 3 (flujos e interdependencias) como columna vertebral del documento.

La segunda es que **la IA como compañera del administrador funciona cuando está bien acotada**. Sin las tres capas de seguridad (whitelist, Human-in-the-Loop y sanitización), el bot sería un riesgo enorme. Con ellas, se convierte en un asistente útil que acelera tareas comunes sin abrir nuevas vías de ataque.

Y la tercera es que **la soberanía tecnológica importa**. La migración a Ollama, el desarrollo de la APK propia y el uso de Tailscale en lugar de soluciones cloud cerradas no son decisiones estéticas: cada una nos ha dado más control sobre el comportamiento del sistema, más privacidad de los datos y menos dependencias externas en producción.

— Unidad Fox Hound, mayo de 2026.